Who am I?

Ziyad Tarek

Sr. SRE/DevOps Engineer @ Rabbit Mart

Certifications

- CKA · CKAD · KCSA · KCNA
- AWS Cloud Practitioner · Terraform Associate
- Google Cloud Associate Cloud Engineer · Huawei Cloud Developer Associate
- RHCSA · GitHub Foundations

Web Servers, Reverse Proxy & Load Balancing

2 Sessions × 2 Hours | Total: 4 Hours

Course Timeline

Session	Content
Session 1	Web Fundamentals, HTTP, Nginx/Apache, Document Roots, Labs 1
Session 2	Load Balancing, Path Routing, Rate Limiting, Caching, Security, API Gateway, Labs 2–4

Session 1

Web Fundamentals & Web Servers

What happens when you type `https://google.com`?

Several layers work together.

```
Browser  →  DNS  →  Internet  →  Web Server  →  App Server  →  Database
```

Components

Component	Role	Examples
Client	Sends requests	Browser, curl, Postman
DNS	Domain → IP	`google.com` → `142.250.191.14`
Web Server	Receives HTTP, returns responses	Nginx, Apache
App Server	Business logic	Node, Django, PHP
Database	Persists data	PostgreSQL, MySQL

HTTP = Hypertext Transfer Protocol

- **Request-response** model
- **Stateless** — each request independent
- **Port 80** (HTTP), **443** (HTTPS)

HTTP Request

```
GET /users HTTP/1.1
Host: example.com
Accept: application/json
Authorization: Bearer eyJhbGc...
```

Part	Purpose
Method	GET, POST, PUT, DELETE
Path	/users
Headers	Auth, content type, cookies

HTTP Methods

Method	Purpose
GET	Retrieve (idempotent)
POST	Create
PUT	Full update
PATCH	Partial update
DELETE	Remove

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json

{"name": "John"}
```

Common Status Codes

Code	Meaning
200	Success
301	Redirect
400	Bad request
401	Unauthorized
403	Forbidden
404	Not found
429	Too many requests
500	Server error

URL Breakdown: `https://example.com/users?id=5`

Part	Name
`https`	Protocol
`example.com`	Host
`/users`	Path
`?id=5`	Query string

URI = identifies resource | **URL** = includes location

HTTP vs HTTPS

	HTTP	HTTPS
Port	80	443
Encryption	None	TLS/SSL
Protects	Nothing	Passwords, cookies, tokens

Certificates: Let's Encrypt, certbot

Static vs Dynamic

Type	Served by	Examples
Static	Web server	HTML, CSS, JS, images
Dynamic	App server	APIs, dashboards, forms

```
User → Nginx → App Server → Database
```

Apache vs Nginx

	Apache	Nginx
Model	Process/thread per request	Event-driven
Config	.htaccess per-directory	Main config only
Best for	Flexibility	Concurrency, reverse proxy

Document Roots — Where Files Live

Server	Path
Apache	`/var/www/html`
Apache (Docker)	`/usr/local/apache2/htdocs/`
Nginx	`/usr/share/nginx/html`
Nginx (custom)	`root /var/www/html;` in config

Request `/` → looks for `index.html` in root

Config Locations

Server	Config path
Apache	`/etc/apache2/sites-available/`
Apache (Docker)	`/usr/local/apache2/conf/httpd.conf`
Nginx	`/etc/nginx/nginx.conf`, `/etc/nginx/conf.d/`

Nginx Config Structure

```
/etc/nginx/
├── nginx.conf       # Main config
├── mime.types       # Content-Type mapping
└── conf.d/          # Site configs
```

Nginx Key Directives

Directive	Purpose
`worker_processes`	# of workers
`root`	Document root
`location /`	Path matching
`proxy_pass`	Forward to backend
`include`	Include config

Apache .htaccess

Per-directory config — no server restart needed.

Use	Example
Redirects	HTTP → HTTPS
Auth	Password-protect dir
Rewrite	Clean URLs
Error pages	Custom 404

Nginx has no .htaccess — use `location` blocks

.htaccess Example

```
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]

AuthType Basic
AuthName "Restricted"
Require valid-user
```

Use: Shared hosting, quick tweaks | **Avoid:** High traffic (move to main config)

Reverse Proxy

```
Client  →  Nginx  →  Backend Servers
```

Does: routing, SSL termination, load balancing, caching, security

Reverse Proxy Config

```
location /api {
    proxy_pass http://backend:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
```

Inspecting HTTP: curl

```
curl -v https://google.com   # Verbose
curl -I https://google.com   # Headers only
```

Shows TLS handshake, headers, status code.

Real Example 1: Rabbitmart API

Embedded dashboard API (JWT in URL + cookies):

```
curl 'https://report.rabbitmart.com/api/embed/card/...' \
  -H 'accept: application/json' \
  -b '_ga=...; metabase.DEVICE=...' \
  -H 'x-metabase-embedded: true'
```

Response: `Token is expired`

Rabbitmart Headers Explained

Header	Purpose
`accept: application/json`	Want JSON
`-b` (cookies)	Auth, analytics
`referer`	Where request came from
`x-metabase-embedded`	Embedding context
JWT in URL	Auth token

Real Example 2: KodeKloud (curl -I)

```
curl -I https://kodekloud.com/
```

Key headers: `cf-cache-status: HIT`, `server: cloudflare`, `x-frame-options: DENY`

KodeKloud Headers

Header	Meaning
`cf-cache-status: HIT`	Served from CDN cache
`age: 58916`	In cache for X seconds
`x-frame-options: DENY`	No iframes (clickjacking)
`strict-transport-security`	Force HTTPS
`x-content-type-options: nosniff`	Prevent MIME sniffing

Lab 1: Basic Web Servers

```
cd webserver-project
docker compose up -d
```

`./web1`, `./web2`, `./web3` → Apache `htdocs/`

```
curl http://localhost:8081
curl http://localhost:8082
curl http://localhost:8083
```

Session 2

Load Balancing, Caching, Security & API Gateway

Load Balancing

```
        Load Balancer
       /      |      \
   Server1  Server2  Server3
```

Benefits: Availability, performance, fault tolerance

Load Balancing Algorithms

Algorithm	Use Case
Round Robin	General purpose
Least Connections	APIs, long-lived
IP Hash	Session persistence
Weighted	Different server capacity

Lab 2: Load Balancer

```
upstream web_cluster {
    server web1:80;
    server web2:80;
    server web3:80;
}
location / { proxy_pass http://web_cluster; }
```

```
cd webserver-project-LB && docker compose up -d
curl http://localhost   # Rotates web1, web2, web3
```

Path-Based Routing

```
/app1 → web1
/app2 → web2
/app3 → web3
```

Lab 3: Path-Based Routing

```
cd webserver-project-Routes
docker compose up -d
```

```
curl http://localhost/app1
curl http://localhost/app2
curl http://localhost/app3
```

`proxy_pass http://web1:80/` — trailing `/` strips path

Rate Limiting

Problem: DDoS, scraping

Solution: Limit requests per IP

```
limit_req_zone $binary_remote_addr zone=req_limit:10m rate=5r/s;
limit_req zone=req_limit burst=3 nodelay;
limit_req_status 429;
```

→ **429 Too Many Requests**

Lab 4: Rate Limiting

```
cd webserver-project-RL
docker compose up -d
```

Trigger: `for i in {1..15}; do curl -s -o /dev/null -w "%{http_code}\n" http://localhost; done`

Expect: mix of 200 and 429

Caching

Problem: Same data, many users → overloaded DB

Solution: Cache at reverse proxy

```
proxy_cache_path /tmp/nginx_cache keys_zone=my_cache:10m;
location /api { proxy_cache my_cache; proxy_pass http://backend; }
```

Benefits: Faster, less backend load

Forward vs Reverse Proxy

	Reverse	Forward
Position	In front of servers	In front of clients
Protects	Servers	Clients
Examples	Nginx, Cloudflare	Corp proxy, VPN

Security Headers

Header	Purpose
`X-Frame-Options: DENY`	No clickjacking
`Content-Security-Policy`	Script/source control
`Strict-Transport-Security`	Force HTTPS
`X-Content-Type-Options: nosniff`	No MIME sniffing

API Gateway

```
Client → API Gateway → User Service
                     → Order Service
                     → Payment Service
```

Does: Auth, routing, rate limiting, logging

Tools: Kong, AWS API Gateway, Traefik

Kong API Gateway

- Nginx + Lua
- JWT, rate limiting, plugins
- Kubernetes, Docker, cloud

Production Architecture

```
Internet → Cloud LB → Reverse Proxy → Microservices → DB + Redis
          (ALB)         (Nginx/Kong)    (K8s)
```

Real-World Scenario: E-Commerce

Nginx LB + 5 app servers + Redis

User → Nginx (cache HIT → instant) | (MISS → app → DB → cache)

Real-World: Embedded Dashboard

iframe + JWT in URL + cookies

→ `referer`, `x-metabase-embedded`, CORS, `frame-ancestors`

Real-World: CDN (KodeKloud)

Cloudflare → HIT = cache | MISS = origin

Headers: `cf-cache-status`, `age`

Real-World: Path Routing (SaaS)

`/tenant-a/*` → app A | `/tenant-b/*` → app B

Same domain, different backends

Real-World: Rate Limiting (API)

100 req/min limit → excess gets 429

Backend stays healthy

Real-World: Corporate Proxy

Employee → Proxy → Internet

Proxy logs, blocks, enforces policy

Internet sees proxy IP, not employee

Lab Summary

Lab	Project	Purpose
1	webserver-project	3 Apache, direct ports
2	webserver-project-LB	Nginx load balancer
3	webserver-project-Routes	Path routing /app1, /app2, /app3
4	webserver-project-RL	Rate limiting (429)

curl Cheat Sheet

Command	Purpose
`curl -v URL`	Verbose
`curl -I URL`	Headers only
`curl -H "X: y" URL`	Add header
`curl -b "c=v" URL`	Cookies
`curl -X POST -d '{}' URL`	POST JSON

Discussion Questions

1. Web server vs application server?
2. Reverse vs forward proxy?
3. Why load balancing?
4. What does caching solve?
5. When do you need an API Gateway?

Thank You

Questions?